Лабораторно упражнение 2

Тема: Типове делегати в език C#. Предикати. Ламбада изрази

I. Теоретична част

В обектно работно пространство System са включени класовете Delegate и MulticatsDelegate, които отговарят за тип делегат. Клас Delegate се явява базов клас за всички типове делегати. Необходимо е да се отбележи, че тези класове могат да бъдат наследявани само служебно от компилатора. Освен тях . NET Core framework съдържа множество типове, които могат да бъдат използвани, когато е необходим делегат. Тези типове използват шаблони в техните дефиниции, като ги прави универсални по отношение на типовете, използвани в делегатите. Такива типове са Action, Func, както и предикатите (Predicate).

1. Делегати от тип Action

Делегатите от тип Action са дефинирани в обектно работно пространство System. Те не връщат резултат, затова могат да сочат (реферират) методи, който са от тип void. Делегатите от този тип могат да имат до 16 аргумента , типовете на аргументите са Generic (шаблонен тип).

Делегати от тип Action са:

```
public delegate void Action();

public delegate void Action<in T>(T arg);

public delegate void Action<in T1, in T2>(T1 arg1, T2 arg2);

.....................
```

Пример:

```
delegate void DisplayMessage(string msg);
        private static void ShowMessage(string msg)
        {
            Console.WriteLine("This is ShowMessage method!");
            Console.Write(msg);
        }
        static void Main(string[] args)
        {
            DisplayMessage messageTarget;
            string s = Console.ReadLine();
            if (s.Length>1)
            {
                messageTarget = ShowMessage;
            }
            else
            {
                messageTarget = Console.WriteLine;
            }
            messageTarget(s);
}
```

В посочения пример се използва делегат `DisplayMessage`, деклариран от програмиста, `messageTarget` е конкретната референция, която ще сочи метода, извеждащ низа на екрана.

Същият примерен код, реализиран с делегат от тип Action е:

```
private static void ShowMessage(string msg)
        {
            Console.WriteLine("This is ShowMessage method!");
            Console.Write(msg);
        }
        static void Main(string[] args)
        {

            Action<string> messageTarget;
            string s = Console.ReadLine();
            if (s.Length>1)
            {
                messageTarget = ShowMessage;
            }
            else
            {
                messageTarget = Console.WriteLine;
            }
            messageTarget(s);
}
```

В случая не е необходимо деклариране на тип делегат, за целта се използва `Action`.

Примерът с използване на ламбда функция е:

```
private static void ShowMessage(string msg)
        {
            Console.WriteLine("This is ShowMessage method!");
            Console.Write(msg);
        }

        static void Main(string[] args)
        {

            Action<string> messageTarget;
            string s = Console.ReadLine();
            if (s.Length>1)
            {
                messageTarget = s=>ShowMessage(s);
            }
            else
            {
                messageTarget = s=>Console.WriteLine(s);
            }
            messageTarget(s);
}
```

Други примери за делегати от тип Action:

```
private static void NewLine()
        {
            Console.WriteLine();
        }
static void Main(string[] args)
{
            Action line = NewLine;
```

```
            line();
            line();
}
```

В случая, делегат `line` от тип `Action` се реферира метод `NewLine`, който отпечатва празен ред. Кодът може да бъде заменен и с ламбда израз:

```
Action line = () => Console.WriteLine();
line();
line();
```

2. Делегати от тип Func

Делегатите от тип Func са делегати от шаблонен тип (generic), които имат един изходен параметър (последен в списъка от параметри). Този тип делегати могат да имат, могат и да нямат входни параметри. Входните параметри са до 16.

```
public delegate TResult Func<out TResult>();

public delegate TResult Func<in T1, out TResult>(T1 arg);

public delegate TResult Func<in T1, in T2, out TResult>(T1 arg1, T2 arg2);

// .................
```

Пример:

```
class Program
{
    static int Sum(int x, int y)
    {
        return x + y;
    }

    static void Main(string[] args)
    {
        Func<int,int, int> add = Sum;
        int result = add(10, 10);
        Console.WriteLine(result);
    }
}
```

Кодът може да бъде заменен и с ламбда израз:

```
Func <int, int, int> add =(x,y) =>x + y;

int result = add(10, 10);

Console.WriteLine(result);
```

Или:

```
Func<int, int, int> add = (int a, int b) =>
              {
                  int sum = a + b;
```

```
                return sum;
            };
int result = add(10, 10);

Console.WriteLine(result);
```

3. Предикати

Предикатите (Predicate) в език C# са делегати подобно на делегатите от тип Func и Action. Те представят метод, който съдържа множество критерии и проверява дали предадения параметър отговаря на тези критерии. Методите, които са от тип предикат трябва да имат един параметър и да връщат резултат от тип bool (true или false).

Делегат от тип Predicate е дефиниран в обектно работно пространство System. Сигнатурата на предиката е следната:

```
public delegate bool Predicate <in T> (T obj);
```

Както и останалите типове делегати, предикатите могат да бъдат използвани с всеки метод, анонимен метод или ламбда израз.

Пример:

```
class Program
    {
        static bool IsUpperCase(string str)
        {
            return str.Equals(str.ToUpper());
        }
        static void Main(string[] args)
        {
            Predicate <string> isUpper = IsUpperCase;

            bool result = isUpper("HELLO !!");
            Console.WriteLine(result);
            result = isUpper("hello !!");
            Console.WriteLine(result);
        }
    }
```

В посочения пример, предикат isUpper се присвоява на метод IsUpperCase, който прави проверка дали всички символи са главни букви. В случая първия върнат резултат е true, а втория – false.

На предикат може да се присвои и ламбда израз, както е показано по-долу:

```
static void Main(string[] args)
        {
            Predicate<string> isUpper = s => s.Equals(s.ToUpper());
            bool result = isUpper("hello world!!");
```

```
            Console.WriteLine(result);
        }
```

Пример 2:

```
private static bool IsPossitive(int b)
{
            if (b >= 0) return true;
            else return false;
}

static void Main(string[] args)
{
      Predicate<int> isPossitive = IsPossitive;
            int n = Int32.Parse(Console.ReadLine());
            result = isPossitive(n);
            Console.WriteLine(result);
}
```

Реализацията чрез ламбда израз е следната:

```
Predicate<int> isPossitive = x=> x >= 0;
int n = Int32.Parse(Console.ReadLine());
result = isPossitive(n);
Console.WriteLine(result);
```

II. Задачи за изпълнение

1. Да се разгледат посочените примери.
2. Като се използват делегати от тип Action да се създаде приложение, което демонстрира работата на аритметичните оператори – методите да извършват действието и да извеждат резултата. Задачата да се реализира по различни начини. Да се използват методи, ламбда функции.
3. Като се използват делегати от тип Function да се създаде приложение, което намира обема на следните фигури: цилиндър, конус, паралелепипед и правилна четириъгълна пирамида с основа правоъгълник . Задачата да се реализира по различни начини. Да се използват методи, ламбда функции.
4. Като се използват предикати да се създаде приложение, което определя дали дадено число е четно или нечетно. Задачата да се различни начини с използване на методи, ламбда функции. Да се въведе масив от числа и да се определи за всяко едно дали е четно или нечетно.